Pressure-driven flow of the viscoelastic Oldroyd-B fluid in narrow non-uniform geometries: re-solved with a physics-informed neural network

Rahul Arvind Masand (2022CH71476) · CLL731 term paper (ML) · IIT Delhi · November 2026

Code, results and report: `https://github.com/RM1102/CLL731-OldroydB-PINN`

---

1. Purpose

Honey flows smoothly; honey with long polymer molecules dissolved in it behaves as if it were full of tiny rubber bands. When such a liquid is squeezed through a narrowing channel, the molecules are stretched and pull back. Boyko & Stone (*J. Fluid Mech.* 936, A23, 2022) asked how this elasticity changes the pressure needed to push a fixed flow rate through a narrow channel that contracts 4:1. They used the Oldroyd-B model and gave formulas valid for weak elasticity (small Deborah number De), checked against computer simulations. Their answer: elasticity *lowers* the pressure drop.

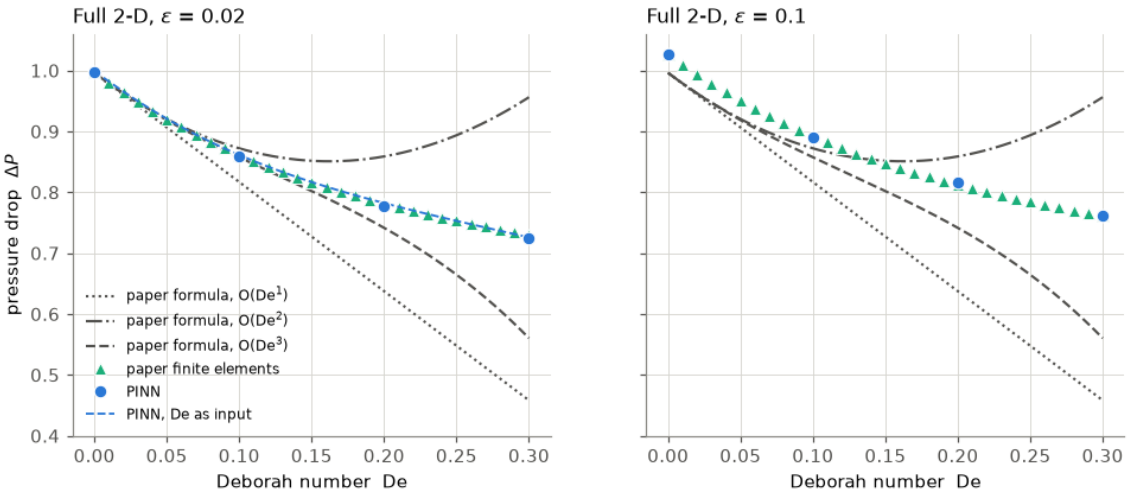
Aim of this project: solve the same equations with a *physics-informed neural network* (PINN), a network trained only to satisfy the flow equations, never shown a solution, and check it carefully against the paper.

2. What was done

1. **Checked the paper.** Re-derived its whole low-De solution with symbolic algebra (SymPy). Every formula is confirmed except one sign typo in arXiv eq. (3.16b).
2. **Recovered the paper's simulation data.** The data were not published, but the figures in the PDF are vector drawings, so the exact data points were read out of the file.
3. **Built an independent reference solver** for the narrow-channel limit. It agrees with the paper's simulations within 0.37 %. A finding on the way: solving station by station down the channel fails for any $\mathrm{De} > 0$, because the stretched molecules and the velocity affect each other in both directions. The whole channel must be solved at once, which a PINN does naturally.
4. **Designed the PINN** so that every boundary condition holds exactly by construction (a stream function for the velocity, symmetric inputs, exact inflow stress). The training loss then contains only the equations. Splitting the pressure into a part along the channel and a small part across it removed a hidden 0.25 % bias.
5. **Trained 16 networks plus one network that takes De as an input**, on a laptop (CPU and its GPU), for the narrow-channel limit and for the paper's two real channels ($\epsilon = 0.02$ and 0.1), at De from 0 to 0.3.

3. Main results

Check	Result	Reference
PINN, narrow-channel limit, $\mathrm{De} \leq 0.3$	within 0.22 %	our reference solver
PINN, full 2-D, both of the paper's channels	within 0.68 %	the paper's simulations
Paper's 3rd-order formula at $\mathrm{De} = 0.3$	23–26 % too low	the paper's simulations
PINN stress fields	0.01–0.5 % over most of the channel	our reference solver



Pressure drop against De for the paper's two channels. Circles: PINN. Triangles: the paper's simulations. Lines: the paper's formulas.

The PINN follows the simulations over the whole range, including $\mathrm{De} = 0.2$–0.3 where the formula fails. The pressure drop falls because of the gradient of the polymer *shear* stress, as the paper says. Errors are largest (a few percent) at the two corners where the wall slope changes suddenly.

4. How to run it (Windows or Mac)

Install Python 3.10–3.13 from python.org (on Windows tick "Add python.exe to PATH"). Then, in a terminal:

Windows: `git clone https://github.com/RM1102/CLL731-OldroydB-PINN.git`
`cd CLL731-OldroydB-PINN` then `setup_and_run.bat`

Mac: same two lines, then `bash setup_and_run.sh`

Without git: on the GitHub page click *Code* → *Download ZIP*, unzip, and double-click `setup_and_run.bat` (Windows) or `setup_and_run_mac.command` (Mac). The script sets up a private Python environment inside the folder and shows a menu:

1) **explorer window**: pick a channel and a field, move the De slider, type a point to read the predicted values;
2) **notebook**: the full report with code and charts (runs in about 20 s using the saved networks);
3) **quick check** (about 10 s);
4) **retrain everything**.

Time warnings. The first setup downloads about 1 GB and takes 5–15 minutes. Retraining takes about **4 hours** on a laptop and is not needed: all trained networks are included. To read the report without installing anything, open `05_notebook/CLL731_OldroydB_PINN.html` in a browser.

5. Using the trained PINN for predictions

A trained PINN is a formula: given a position and De, it returns velocity, pressure and polymer stress in a fraction of a second, without solving anything again. Besides the explorer window, `predict.py` answers from the command line, for example `python predict.py --case eps0.02 --De 0.2 --dp` for the pressure drop (also in pascals for the paper's channel), `--point 0.5 0.3` for every field at a point, or `--csv out.csv` for a whole field. Valid range: the paper's channel ($\alpha = 4$, $\tilde{\beta} = 0.4$) and $0 \leq \mathrm{De} \leq 0.3$.

6. Limits and acknowledgement

The PINN takes minutes to train per case, where our reference solver takes seconds; its strengths are exact boundary conditions, no mesh, and one network that answers any De instantly. Accuracy is limited at the two corners. Higher De (> 0.5), where Oldroyd-B stresses grow without bound, was not attempted. *Use of AI:* the literature search, code and text were developed with the help of an AI assistant (Claude), as stated in the project proposal; all results were checked against the paper and an independent solver.